

# UNIT-3 | Lecture 3

## Aggregate Operators, NULL Values & Complex Integrity Constraints

**Coverage:** Hour 5

### Learning Objectives

Use aggregate functions and GROUP BY/HAVING.

Understand NULL and three-valued logic.

Understand complex integrity rules.

### Aggregate Operators

**COUNT:** counts rows or non-NULL values depending on the expression.

**SUM:** total of non-NULL numeric values.

**AVG:** average of non-NULL numeric values.

**MIN / MAX:** minimum and maximum values.

```
SELECT Branch, COUNT(*) FROM Student GROUP BY Branch;
```

### GROUP BY & HAVING

**GROUP BY** forms groups for aggregate calculations.

**HAVING** filters groups after aggregation.

```
SELECT Branch, AVG(Marks)
FROM Student GROUP BY Branch
HAVING AVG(Marks) > 70;
```

### NULL Values

**NULL** represents unknown, missing, or not-applicable information; it is not the same as zero or an empty string.

Use `IS NULL` or `IS NOT NULL`.

SQL uses three-valued logic: **TRUE**, **FALSE**, **UNKNOWN**. Comparisons involving NULL generally evaluate to UNKNOWN.

### Complex Integrity Constraints

SQL can combine **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **NOT NULL**, **CHECK**, and referential actions to enforce database rules.

Examples of referential actions include **ON DELETE CASCADE** and **ON DELETE SET NULL**, depending on DBMS support.

Some complex business rules may require triggers.

### Exam Focus

**COUNT(\*)** counts rows; **COUNT(column)** ignores NULL values in that column.

Never use `= NULL`; use `IS NULL`.

### Key Takeaway

Aggregates summarize data, NULL represents missing/unknown information, and constraints protect database consistency.